

PLE Project

Manuela Victoria Ragua - 202310825

Laura Valentina Fonseca Rincón - 202222197

A partir del enunciado, se define la siguiente gramática para el lenguaje Verilang

1) Terminales:

a. Palabras reservadas:

{ "defmodule", "using", "defspace", "defrule", "end", "defoperator",
"defexpression", "defvar", "forall", "exists", "defer", "in" }

b. Operadores y simbolos:

{ ":", ":", ":", "(", ")", "[", "]", "<", ">", "<=", ">=", "<>", "=", "and", "or", "neg", "≡",
"=>", "**", "/", "-", "+", "**", "%", "->" }

c. Numeros:

{ "1"; "2"; "3"; "4"; "5"; "6"; "7"; "8"; "9"; "0" }

d. Letras:

{ "A"; "B"; "C"; "D"; "E"; "F"; "G"; "H"; "I"; "J"; "K"; "L"; "M"; "N"; "O"; "P"; "Q"; "R"; "S"; "T";
"U"; "V"; "W"; "X"; "Y"; "Z"; "a"; "b"; "c"; "d"; "e"; "f"; "g"; "h"; "i"; "j"; "k"; "l"; "m"; "n";
"o"; "p"; "q"; "r"; "s"; "t"; "u"; "v"; "w"; "x"; "y"; "z" }

2) No terminales:

{ Module, ImportList, Import, ModuleBody, Definition, SpaceDef, OperatorDef,
VarDef, VarList, VarDecl, RuleDef, ExpressionDef, Expression,
QuantifiedExpression, Quantifier, OperatorApplication, OperatorSignature,
Type, AttributeList, Attribute, Identifier, PrefixApplication, ArgumentList,
InfixApplication, SimpleTerm, InfixOperator, EquivalenceExpression,
ImplicationExpression, OrExpression, AndExpression,
ComparisonExpression, PrimaryExpression, IntLiteral, FloatLiteral, Number,
CharLiteral, ComparisonOp }

3) Simbolo inicial: Module

4) Reglas de producción:

a. Programa y Módulos:

```
Module ::= "defmodule" Identifier ImportList ModuleBody "end";  
ImportList ::= {Import};  
Import ::= "using" Identifier;
```

b. Cuerpo del modulo

```
ModuleBody ::= {Definition};  
Definition ::= SpaceDef | OperatorDef | VarDef | RuleDef |  
ExpressionDef;
```

c. Definición de espacios

```
SpaceDef ::= "defspace" Identifier [ "<" Identifier ]  
"end";
```

d. Definición de Operadores

```
OperatorDef ::= "defoperator" Identifier ":" OperatorSignature [ AttributeList ]  
"end";  
OperatorSignature ::= Type "->" Type { "->" Type };  
Type ::= Identifier ;
```

e. Atributos

```
AttributeList ::= "[" Attribute { Attribute } "]" ;  
Attribute ::= Identifier | Identifier ":" Identifier ;
```

f. Variables

```
VarDef ::= "defvar" VarList "end";  
VarList ::= VarDecl { "," VarDecl } ;  
VarDecl ::= Identifier ":" Type ;
```

g. Reglas

```
RuleDef ::= "defrule" OperatorApplication "->" OperatorApplication  
"end";
```

h. Aplicación de operadores:

OperatorApplication ::= PrefixApplication | InfixApplication ;

PrefixApplication ::= "(" Identifier ArgumentList ")" ;

ArgumentList ::= SimpleTerm { SimpleTerm }

InfixApplication ::= SimpleTerm InfixOperator SimpleTerm ;

SimpleTerm ::= Identifier | "(" Expression ")" ;

InfixOperator ::= Identifier | "in" ;

i. Expresiones

ExpressionDef ::= "defexpression" Expression "end";

j. Expresiones Lógicas

Expression ::= QuantifiedExpression | EquivalenceExpression ;

EquivalenceExpression ::= ImplicationExpression |
ImplicationExpression EquivalenceOp EquivalenceExpression ;

EquivalenceOp ::= "==" | "≡" ;

ImplicationExpression ::= OrExpression | OrExpression "=>"
ImplicationExpression ;

OrExpression ::= AndExpression | AndExpression "or" OrExpression ;

AndExpression ::= ComparisonExpression | ComparisonExpression "and"
AndExpression ;

ComparisonExpression ::= PrimaryExpression | PrimaryExpression
ComparisonOp PrimaryExpression ;

PrimaryExpression ::= OperatorApplication | Identifier | "(" Expression ")" ;

ComparisonOp ::= "=" | "<" | ">" | "<=" | ">=" | "<>";

k. Cuantificadores

QuantifiedExpression ::= Quantifier Identifier "in" Type "." Expression
;

Quantifier ::= "forall" | "exists";

l. Escritura

IntLiteral ::= Number {Number};

FloatLiteral ::= Number{Number} "." Number{Number};

m. Numero

Number ::= "1"|"2"|"3"|"4"|"5"|"6"|"7"|"8"|"9"|"0"

n. Character

CharLiteral ::=

"A"|"B"|"C"|"D"|"E"|"F"|"G"|"H"|"I"|"J"|"K"|"L"|"M"|"N"|"O"|"P"|"Q"|"R"|"S"|"T"|"U"|"V"|"W"|"X"|"Y"|"Z"|"a"|"b"|"c"|"d"|"e"|"f"|"g"|"h"|"i"|"j"|"k"|"l"|"m"|"n"|"o"|"p"|"q"|"r"|"s"|"t"|"u"|"v"|"w"|"x"|"y"|"z"

o. Identificador

Identifier ::= CharLiteral { CharLiteral | Number | "-" };

5) Cambios realizados segun la entrega anterior

En primer lugar, se corrigió la definición de Identifier, ya que antes existía una inconsistencia entre su uso como terminal y no terminal. En la versión actual, se definió como un elemento léxico, evitando conflictos en el análisis sintáctico. También se tuvo en cuenta la restricción de palabras reservadas, evitando que estas sean reconocidas como identificadores.

Luego, se reemplazó la definición de Expression por una jerarquía de precedencia. Se estructuraron las expresiones en distintos niveles: EquivalenceExpression, ImplicationExpression, OrExpression, AndExpression, ComparisonExpression y PrimaryExpression, lo que permite interpretar correctamente los operadores y elimina ambigüedades en el parsing.

También se corrigió la definición de OperatorSignature para permitir el uso de tipos simples encadenados mediante el operador $\rightarrow$. Esto garantiza que las firmas de los operadores sean consistentes con la semántica.

También se ajustó la definición de ArgumentList restringiéndola a una secuencia de SimpleTerm lo cual simplifica la gramática y evita ambigüedades de operadores.

Por otra parte, se agregó una definición explícita para los operadores de equivalencia mediante EquivalenceOp, permitiendo tanto el operador ASCII $==$ como el operador Unicode $\equiv$.

También se revisó la definición de operadores lógicos y de implicación, asegurando consistencia entre la gramática y su implementación, teniendo en cuenta que en Rascal algunos símbolos requieren escape $\backslash$ como $\backslash >$.

Finalmente, se añadieron definiciones léxicas formales para `IntLiteral`, `FloatLiteral` y `CharLiteral`, las cuales no estaban especificadas en la versión anterior. Esto mejora el reconocimiento de literales y garantiza una correcta tokenización del lenguaje.